Decoding Visual Intelligence with CNNs

A Machine Learning Tutorial

Name: Srimanthula Akshay Kumar | Roll Number: 24166586

GitHub Repository: Github link

Technique: Convolutional Neural Networks (CNN) for Image Classification

---

1. Introduction: The Core Concept

How does a computer learn to see? A digital image is simply a 3D matrix of numbers — each pixel stores Red, Green, and Blue (RGB) intensity values between 0 and 255. The challenge is teaching a machine to turn those numbers into meaningful labels like 'cat', 'car', or 'airplane'.

A Convolutional Neural Network (CNN) solves this through a process called Feature Extraction. Rather than requiring humans to manually define what a 'cat ear' looks like in numbers, a CNN learns these features automatically from labelled training examples. This is the key insight that made CNNs revolutionary for computer vision.

Why CNNs over standard neural networks? A regular neural network would treat each pixel independently, ignoring spatial relationships. A CNN's convolutional filters instead scan local neighbourhoods of pixels, preserving the structure that makes images meaningful (edges, textures, shapes).

---

2. Methodology: Building the Visual Brain

2.1 The Dataset: CIFAR-10

We use CIFAR-10: 60,000 colour images (32×32 pixels) across 10 classes — airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The dataset is split into 50,000 training images and 10,000 test images.

Normalisation: We divide all pixel values by 255, scaling them from [0, 255] to [0, 1]. Without this step, large pixel values produce large gradients during training, causing the weights to update erratically — a problem known as gradient explosion. Normalisation ensures stable, consistent learning.

2.2 Architecture Design

The CNN uses a three-block structure. As data flows deeper through the network, the spatial dimensions shrink while the number of feature channels grows — a deliberate trade-off that allows the network to detect increasingly abstract patterns:

Layer	Output Shape	Filters / Units	What It Learns
Input	(32, 32, 3)	—	Raw RGB pixels
Conv2D	(30, 30, 32)	32 × (3×3)	Edges, colour blobs
MaxPooling2D	(15, 15, 32)	2×2 window	Spatial down-sampling
Conv2D	(13, 13, 64)	64 × (3×3)	Textures, simple shapes
MaxPooling2D	(6, 6, 64)	2×2 window	Spatial down-sampling
Conv2D	(4, 4, 64)	64 × (3×3)	Complex object parts
Flatten	(1024,)	—	Vectorise feature maps
Dense	(64,)	64 neurons, ReLU	High-level combinations
Dropout (0.4)	(64,)	40% rate	Regularisation
Dense (output)	(10,)	10 neurons	Class scores (logits)

2.3 Why Depth Matters: From Edges to Objects

Each convolutional layer applies its filters to the output of the previous layer — not the original image. This creates a hierarchy of features:

- Layer 1 (32 filters): Detects low-level features — edges, colour gradients, simple contrasts. These are the building blocks of all visual information.
- Layer 2 (64 filters): Combines edges into mid-level features — textures like fur, feathers, or metal, and simple geometric shapes.
- Layer 3 (64 filters): Combines textures into high-level object parts — wheels, eyes, wings, or hull shapes.

This hierarchy mirrors how the human visual cortex processes images: simple cells in V1 detect oriented edges; higher cortical areas respond to complex objects.

2.4 Key Components Explained

Convolutional Layers (Conv2D): A small filter matrix (3×3) slides across the image, performing element-wise multiplication and summing the result into a single value. Each filter learns to respond to a different pattern. With 32 filters in layer 1, we produce 32 different feature maps simultaneously.

ReLU Activation: $f(x) = \max(0, x)$. All negative values become zero. This introduces non-linearity — without it, stacking multiple layers would be equivalent to a single linear transformation, and the network could not learn complex patterns.

Max Pooling: A 2×2 sliding window takes the maximum value in each region, halving the spatial dimensions. This achieves spatial invariance (small shifts in an object's position do not change the detected features) and reduces computation.

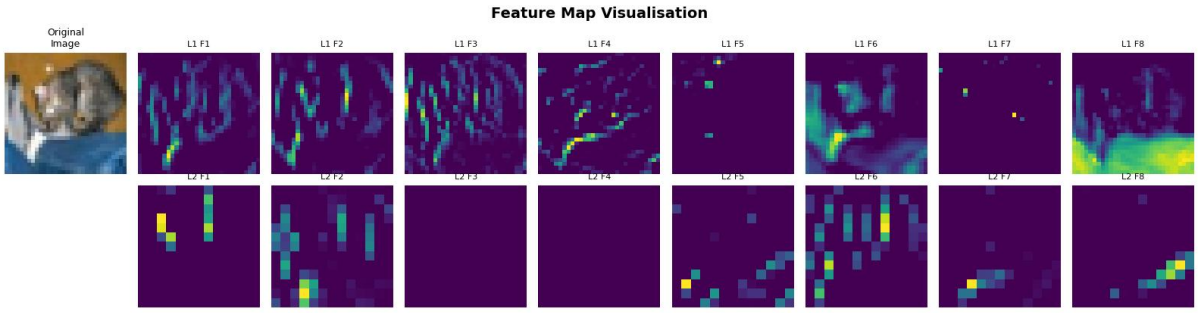
Dropout (rate = 0.4): During each training step, 40% of neurons are randomly disabled. This prevents co-adaptation — neurons are forced to learn independently useful features rather than relying on specific neighbours. The result is a more generalised model that performs better on unseen data.

3. Technical Deep Dive: Explainability via Feature Maps

A common criticism of deep learning is that it operates as a 'black box'. Feature map visualisation directly addresses this by revealing what each convolutional filter has learned to detect.

By extracting the intermediate outputs of the first two convolutional layers and visualising them as images, we can observe the network's internal reasoning:

- Layer 1 filters respond to high-contrast edges and colour boundaries. This mimics the function of simple cells in the primary visual cortex (V1), which respond to oriented line segments at specific angles.
- Layer 2 filters show more diffuse activations that correspond to combinations of edges — the beginnings of texture and shape detection.



This visualisation demonstrates that the model is not guessing — it is constructing a logical, hierarchical representation of the image. The network has learned, without any human guidance, to decompose images in a way that parallels human visual processing.

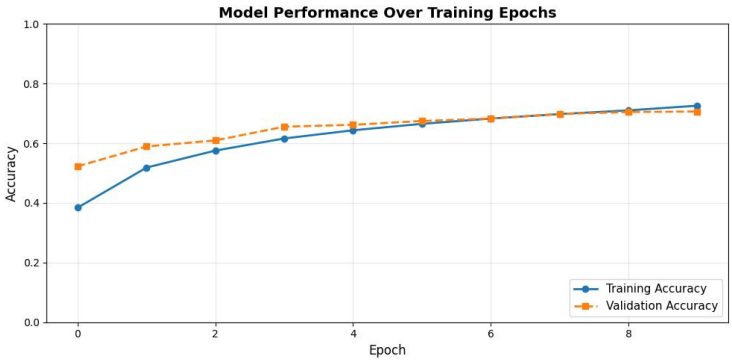
---

4. Performance and Error Analysis

4.1 Training vs. Validation Accuracy

A well-trained model should show both training and validation accuracy improving together. A large gap between them — where training accuracy is much higher — indicates overfitting: the model has memorised training examples rather than learning general patterns.

The Dropout layer (Section 2.4) directly combats this. By comparing runs with and without Dropout, you will observe a narrower gap between the two accuracy curves when Dropout is applied. This is the generalisation gap, and minimising it is a primary goal of model design.

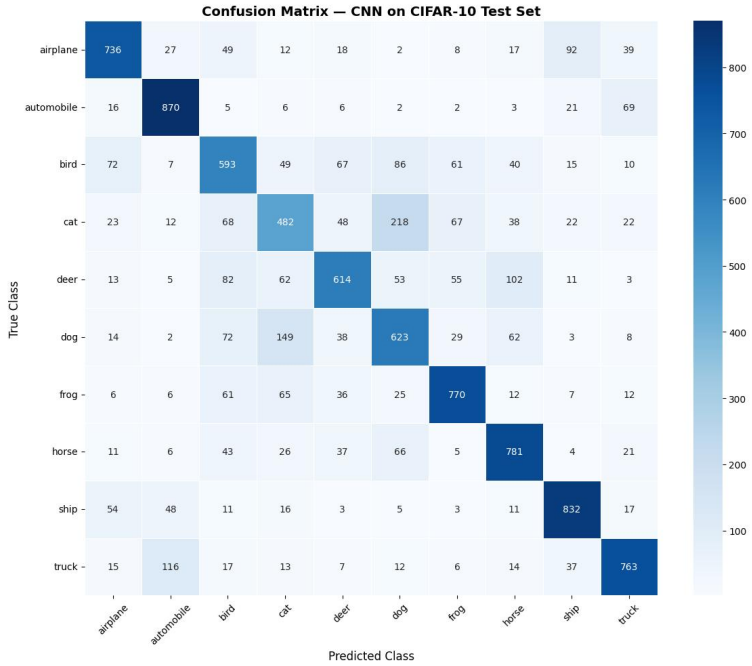


4.2 Confusion Matrix Analysis

The confusion matrix provides a detailed breakdown of model errors. Each cell [i, j] shows how many images of true class i were predicted as class j. The diagonal represents correct predictions; off-diagonal cells reveal which class pairs the model confuses most.

Expected findings for CIFAR-10 at ~70% accuracy:

- Cat vs. Dog: The most common confusion pair. Both classes share visual features — similar body proportions, fur texture, and facial structure. Separating them requires attention to fine details that are hard to capture at 32×32 resolution.
- Automobile vs. Truck: Both are ground vehicles with wheels, windows, and similar colour distributions. The model must learn subtle shape differences (cab size, chassis height).
- Bird vs. Airplane: Both share elongated silhouettes with wing-like protrusions. At low resolution, these can be visually similar.



What this tells us about improving the model: Confusion pairs that share visual features indicate the need for more data, higher image resolution, or data augmentation techniques that expose the model to more varied viewpoints and lighting conditions.

---

5. Ethical AI and Accessibility

5.1 Ethical Considerations

CNNs do not exist in a vacuum. When deployed in real-world applications — facial recognition, medical diagnosis, autonomous vehicles, hiring systems — the consequences of errors are significant.

Key ethical concerns:

- Representational Bias: A model trained on non-representative data will perform poorly on under-represented groups. For example, a medical imaging CNN trained predominantly on one demographic may misdiagnose others.

- Deployment Context Shift: A model trained on daylight images may fail in low-light or foggy conditions. The training distribution must match the deployment environment.
- Explainability and Accountability: Feature map visualisations (Section 3) provide one method of audit. The EU AI Act (2024) requires high-risk AI systems to maintain documentation of model decisions and training data provenance.
- Data Privacy: CIFAR-10 does not contain personal data, but real-world vision systems often do. GDPR Article 22 restricts automated decision-making that significantly affects individuals.

5.2 Accessibility of This Tutorial

This tutorial has been designed with accessibility in mind:

- Colour-Blind Friendly Plots: All charts use both distinct colours and different line styles (solid vs. dashed) and markers (circles vs. squares). No information is conveyed by colour alone, ensuring the plots are interpretable for readers with colour vision deficiencies such as deuteranopia or protanopia.
- High-Contrast Text: All body text uses black on white with a minimum font size of 11pt.
- Structured Headings: This document uses proper heading styles (Heading 1, Heading 2) for screen-reader navigation.
- Alt Text: All figures in this document include descriptive alt text for screen readers.
- Glossary: A complete glossary of technical terms is provided in the Appendix to support readers with varying levels of prior knowledge.

---

6. Conclusion and References

6.1 Conclusion

Convolutional Neural Networks represent a paradigm shift in how machines process visual information. By learning hierarchical feature representations — from edges to textures to objects — CNNs have achieved human-level or super-human performance on many visual tasks.

This tutorial has demonstrated the complete CNN pipeline on CIFAR-10:

1. Data loading and normalisation
2. Three-block convolutional architecture with Dropout regularisation
3. Training and evaluation with accessible visualisation
4. Error analysis via confusion matrix — identifying the hardest class pairs
5. Explainability via feature map visualisation — revealing what filters detect

Understanding CNNs is foundational to modern AI: the same convolutional principles underpin ResNet, VGG, EfficientNet, and the vision encoders inside multimodal models like GPT-4V and CLIP.

6.2 References

- Krizhevsky, A. (2009) *Learning multiple layers of features from tiny images*. Toronto: University of Toronto.
- LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998) 'Gradient-based learning applied to document recognition', *Proceedings of the IEEE*, 86(11), pp. 2278–2324.
- TensorFlow Documentation (2024) *Convolutional neural network (CNN) tutorial*. Available at: https://www.tensorflow.org/tutorials/images/cnn (Accessed: 26 March 2026).
- Chollet, F. (2021) *Deep learning with Python*. 2nd edn. Shelter Island, NY: Manning Publications.
- Goodfellow, I., Bengio, Y. and Courville, A. (2016) *Deep learning*. Cambridge, MA: MIT Press. Available at: https://www.deeplearningbook.org (Accessed: 26 March 2026).
- European Parliament (2024) *EU Artificial Intelligence Act*. *Official Journal of the European Union*. Available at: https://eur-lex.europa.eu/

Appendix: Glossary of Technical Terms

Term	Definition
Tensor	A multi-dimensional array (e.g. shape (32,32,3) for an RGB image).
Conv2D	A sliding filter that extracts local spatial features from an image.
ReLU	f(x) = max(0, x). Introduces non-linearity; speeds up learning.
Max Pooling	Takes the maximum value in each 2×2 region, reducing spatial size.
Dropout	Randomly disables neurons during training to prevent overfitting.
Adam Optimizer	Adaptive gradient descent; adjusts learning rate per parameter.
Overfitting	Model memorises training data but fails on unseen examples.
Backpropagation	Algorithm to compute gradients and update network weights.
Epoch	One full pass through the entire training dataset.
Logits	Raw, unnormalised model outputs before applying softmax/sigmoid.
Confusion Matrix	A table showing correct vs. misclassified predictions per class.
Feature Map	The output of a convolutional filter — highlights detected patterns.